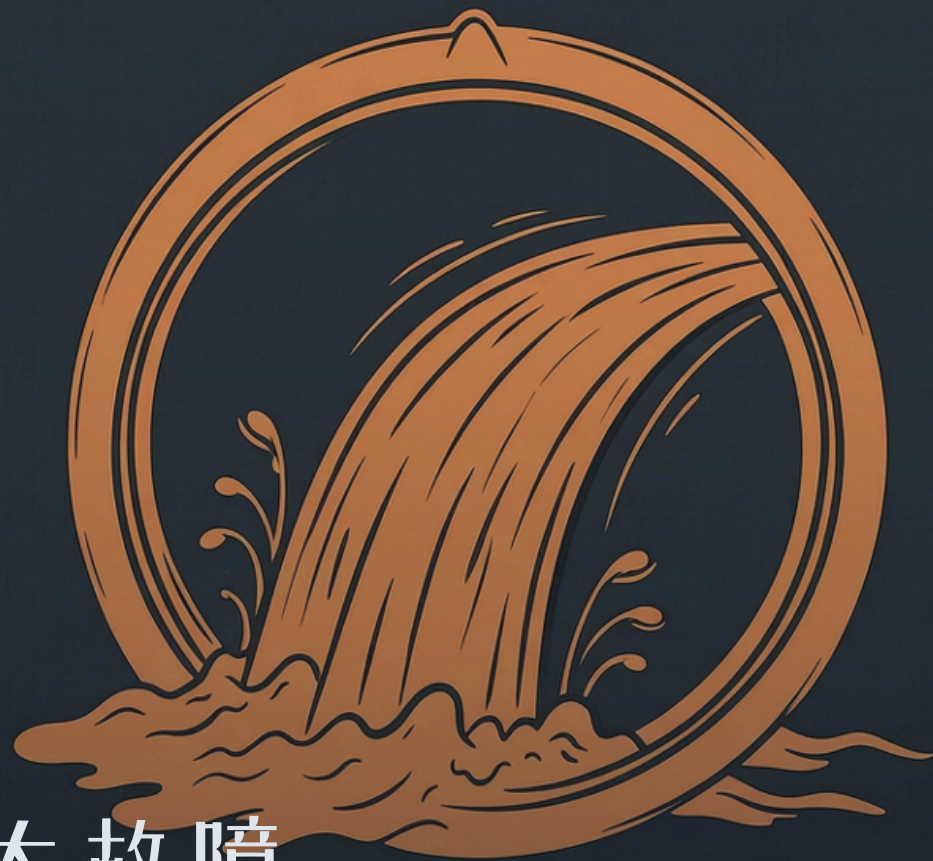




Redis 缓存三大故障

缓存是数据库前的一道防线，一旦失守后果严重

缓存的承诺与脆弱性

缓存的核心价值是替数据库挡住请求，但这个承诺在三种场景下会失效。

穿透

查不到的数据

请求的数据既不在缓存也不在数据库，缓存形同虚设，每次请求都穿透到 DB

击穿

过期的热点

某个高并发访问的热点 Key 突然过期，大量并发请求在同一瞬间穿透缓存涌入数据库

雪崩

同时到期的 Key

大量缓存在同一时间集中过期或 Redis 节点宕机，导致请求在短时间窗口内全面回源

Client
发起读请求

Redis
命中返回 · 失效穿透

DB
承受回源压力

缓存穿透：查不存在的数据

缓存穿透是指请求的数据既不在缓存也不在数据库，缓存形同虚设，每次请求都穿透到 DB。

无防护



每次请求都回源查询，DB 持续承压甚至被拖垮

有防护



布隆过滤器拦截 / 缓存空值命中，直接返回，DB 不受影响

1 参数校验

在入口层拦截非法 key，如空值、超长字符串、明显不存在的 ID，避免无效请求进入缓存层

2 布隆过滤器

在缓存前加一道快速判存屏障，误判率可控（如 1%），但存在误判且不能删除元素，重建成本高

3 缓存空值

对查不到的结果也写缓存并设短 TTL（如 30 秒），内存代价可控，但攻击者可换 key 绕过

缓存击穿：热点 Key 过期的瞬间



缓存击穿是指某个高并发访问的

热点 Key 突然过期，大量并发请求在同一瞬间穿透缓存涌入数据库。与穿透的区别：**数据存在，只是过期那一刻的并发风暴**。

方案一：互斥锁

使用 Redisson 分布式锁，`tryLock(100, 30, TimeUnit.MILLISECONDS)` 最大等待 100ms，持有锁 30s 自动释放，支持看门狗续期。将并发转为串行，吞吐代价是未获取锁的线程需等待。

方案二：逻辑过期

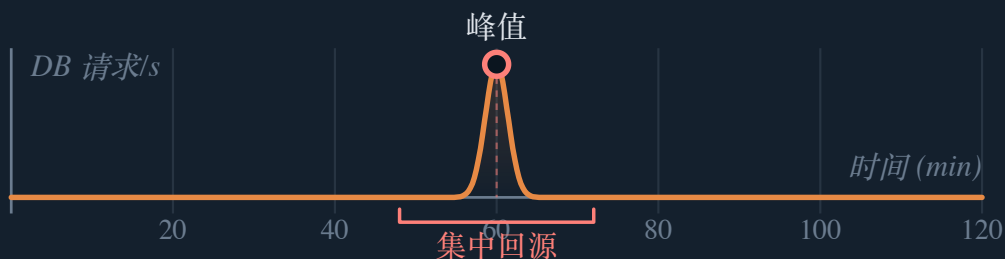
缓存不设 TTL，由业务代码判断是否过期并异步重建。重建期间返回旧数据，牺牲数据一致性换取高可用，适合对一致性要求不高的场景。

缓存雪崩：大量 Key 同时失效

大量缓存集中过期或 Redis 节点宕机，请求在短时间窗口内全面回源，数据库承受不住。

与击穿的区别：不是一个 Key，是一片 Key 同时失效。

// 集中过期 — 危险



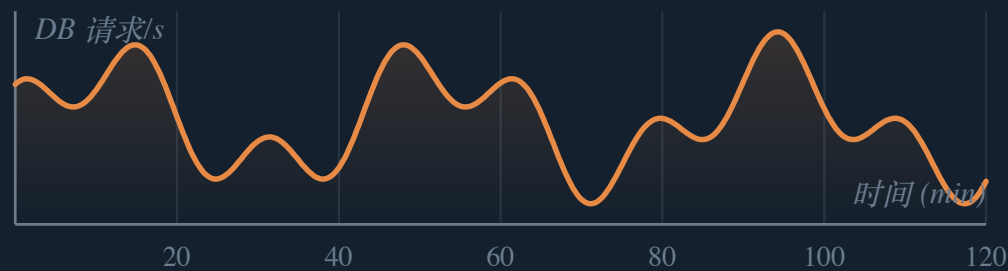
● 预防层

TTL 随机偏移 — 基础 30min + 随机 0~10min，打散过期时间

多级缓存 — 本地缓存 + Redis，层层拦截回源请求

高可用集群 — 哨兵 / Cluster，避免单节点宕机引发雪崩

// 随机打散 — 平缓



● 兜底层

限流 — 超过阈值直接返回繁忙或降级数据

降级 — 返回默认值 / 旧数据 / 关闭非核心功能

熔断 — DB 压力过高时快速失败，不拖垮全链路

三者对比：一张图分清



维度	穿透	击穿	雪崩
根因	数据不存在	热点 Key 过期	大批量 Key 过期 / Redis 宕机
影响范围	单个 key 持续穿透	单个热点 key 瞬间峰值	大批量 key 持续洪峰
DB 压力模式	持续穿透 · DB 长期承压	瞬间峰值 · DB 短时过载	持续洪峰 · DB 可能被拖垮
核心解法	布隆过滤器 + 缓存空值	互斥锁 + 逻辑过期	TTL 随机化 + 限流降级熔断

实战防御：分层纵深策略

从工程实践角度，一个完整的缓存防御体系应该分四层搭建——不是四选一，而是每一层解决不同问题，纵深防御才可靠。

● 入口层 第一道屏障

参数校验拦截非法 key，布隆过滤器在缓存前加一道快速判存屏障，拦截不存在的请求

● 缓存层 核心策略

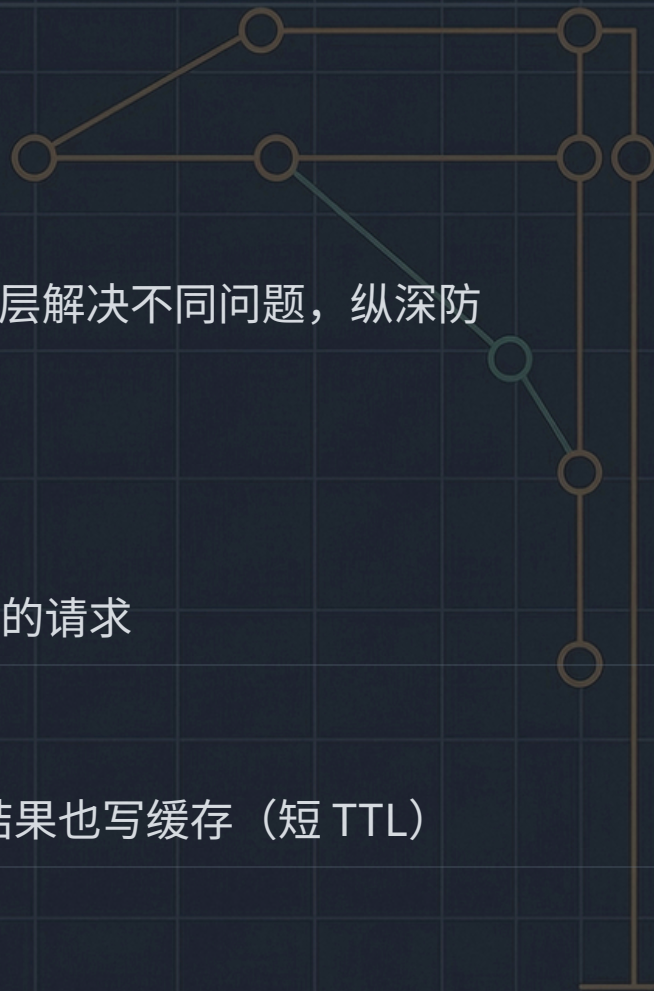
TTL 随机化打散过期时间，热点 Key 永不过期 + 逻辑过期异步重建，查不到的结果也写缓存（短 TTL）

● 高可用层 基础设施

哨兵模式或 Cluster 集群保障 Redis 自身不成为单点，多级缓存架构（本地缓存 + Redis）

● 兜底层 最后防线

限流、降级、熔断保护数据库，当 DB 压力过高时快速失败而非雪崩式拖垮全链路



记住三句话



穿透

挡

布隆过滤器

在请求到达缓存之前，**拦住不存在的 key**，快速失败，不回源。

击穿

等

互斥锁

并发排队，**只允许一个线程回源**重建缓存，其余等待结果。

雪崩

散

随机 TTL + 限流

随机过期时间**打散洪峰**，加上限流降级熔断兜底。